LLM-Based Anomaly Detection in System Logs

Lam Christopher Nguyen
la815794@ucf.edu
University of Central Florida
Orlando, Florida, USA

Abstract

In this project, LLMs will be applied in a software application to identify abnormal patterns in network and system logs. The practical application that was deveoped to test LLM anomaly detection is a web-based system designed using the MongoDB, Express, React, and Node (MERN) stack. The framework that will be used to scan the network logs is called LangChain combined with Claude. The topic number in this syllabus is: 14. LLM-Based Anomaly Detection in System Logs. So far the application has been built with Backend Logging using the Javascript logging Library Winston which so far is able to log backend API requests and responses. LangChain framework has been implemented to some extent and is able to scan logs. However more fine-tuning needs to be done as it is not consistent and there are still bugs and issues with the Claude API.

CCS Concepts

• **Computing methodologies** → **Natural language processing**; *Neural networks*; • **Security and privacy** → **Intrusion/anomaly detection and malware mitigation**; • **Software and its engineering** → System administration.

Keywords

large language models, anomaly detection, log analysis, system monitoring, LangChain, natural language processing, security analysis, network logs

ACM Reference Format:
Lam Christopher Nguyen. 2026. LLM-Based Anomaly Detection in System Logs. In . ACM, New York, NY, USA, 3 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Code Availability

The source code for the implementation of this web application can be found at:

https://github.com/lamcnguyen89/CAP_6640_AI_Logging

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, Washington, DC, USA
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

2 Introduction and Problem Statement

2.1 Motivation

The motivation for this project is to explore how LLMs can be used to identify problems in system and network logs, which can be a valuable tool for system administrators and security professionals.

2.2 Significance

The significance of this project is that it can help to improve the efficiency and effectiveness of log analysis, which can lead to faster identification of issues and improved security.

2.3 Objectives

The objectives of this project are to develop a web application that can scan network logs using LangChain and Claude, and to evaluate the performance of the application in identifying anomalies in the logs.

3 Related Work

Recent advances in applying large language models to log analysis have shown promising results.

Egersdoerfer et al. [1] began using LLMs such as ChatGPT for log analysis since they are able to retain context and formulate insightful responses over conversations. These LLMs are able to conduct few-shot and in-context learning with reasoning ability. Previously there were several problems with pre-existing techniques. The first issue was that there was a heavy reliance on expert-labeled logs to discern anomalous patterns. This manual labelling slowed down training. The second issues was that they relied on numeric model prediction based on numeric vector input which is not human readable. This lack of human readability rules out targeted error correction by humans.

Qi et al. [4] introduced LogGPT. This paper explored the use of ChatGPT for log-based anomaly detection. It demonstrated that ChatGPT could be effectively used to identify anomalies in system logs, showing promising results in terms of accuracy and efficiency compared to traditional methods.

Guan et al. [2] introduced LogLLM.Traditional deep-learning methods often struggle to capture semantic information in log messages which are typically organized in natural language. LogLLM employed BERT for extracting semantic vectors from log messages, and uses Llama for classifying log sequences. It also uses a projector to align the vector spaces of BERT and Llama to ensure consistency in understanding log semantics. LogLLM doesn't require log parsers to extraxt templates from logs.

He et al. [3] introduced LLMeLog. This paper introduced a system intended to solve two problems. The first problem is that semantic noise is buried in log events which lead to inevitable gaps between the semantics from the logs and their actual intended meaning.

The second problem was that there exists a gap between general semantic embeddings and the specific embedding requirement for anomaly detection tasks. LLMeLog uses LLMs to enrich the content of log events with in-context learning techniques. These enriched logs are then used to fine-tune a pre-trained BERT model. Finally, it trains a transformer-based anomaly detection model with the event representationas produced by this fine-tuned BERT model. F1 scorese exceeded 99 percent and even with using only 10 percent of the labeled data, was still able to achieve an F1 score of 90 percent.

4 Methodology

In order to demonstrate how LLMs can be used to identify problems in system and network logs, a web application will be developed using the MERN stack. The application will then utilize some LLM log frameworks that were developed for LLM log analysis to scan a repository of logs and identify anomalies. An attempt will be made to balance performance with cost efficiency. The performance of the application will be evaluated using standard evaluation metrics for anomaly detection.

4.1 Techniques

The technique for working with Logging involves 4 steps.

The first step is to collect and possibly parse the logs. The way the application is designed, all the logs will be generated and downloaded into a single location. The logs will then be parsed and preprocessed to ensure that they are in a format that can be easily analyzed by the LLM.

The second step is to perform Vector Search and Contextual Retrieval. This step provides a baseline of what normal operations shown in the logs will look like. Retrieval augmented generation (RAG) is a technique that can be used to improve the performance of LLMs by providing them with relevant context from the logs. This can be done using vector search techniques to retrieve relevant log entries based on their semantic similarity to the input query. RAG will be used to feed the LLM.The LLM that will be used is Claude.

The third step is to perform Anomaly Detection. This step involves using the LLM to analyze the logs and identify any anomalies or patterns that may indicate a problem. The LLM can be trained to recognize normal patterns in the logs and flag any deviations from those patterns as potential anomalies.

The fourth step is to analyze the root cause of all the errors from the series of logs and explain in human readable language what the problem is and how to fix it.

4.2 Evaluation Metrics

The performance of the application will be evaluated using standard evaluation metrics for anomaly detection, such as precision, recall, and F1 score. These metrics will be calculated based on the number of true positives, false positives, and false negatives identified by the application.

For review, precision is the ratio of true positives to the total number of positive predictions made by the model. Recall is the ratio of true positives to the total number of actual positive cases in the dataset. The F1 score is the harmonic mean of precision and recall, providing a single metric that balances both aspects of performance.

5 Evaluation and Results

So far the application has been built with Backend Logging using the Javascript logging Library Winston which so far is able to log backend API requests and responses. LangChain framework has been implemented to some extent and is able to scan logs. However more fine-tuning needs to be done as it is not consistent and there are still bugs and issues with thee Claude API.

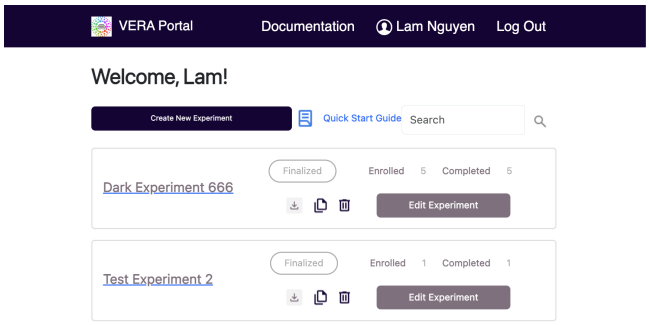

Figure 1: Application UI

You can see in Figure 1 the user interface of the application. The application is essentially a web application that keeps track of experiments and experiment data. It is a standard web application with Create, Read, Update and Delete functionality. Data is stored in a database and logs are generated everytime there is communication between the front end of the code which is the website and the backend server.

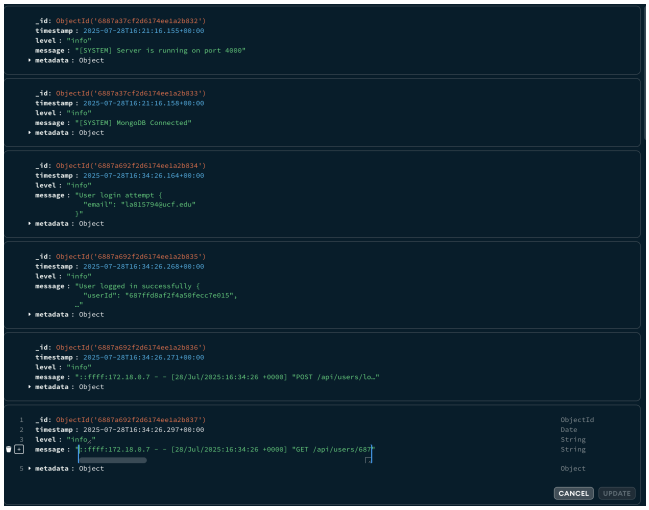

Figure 2: Backend Logs saved in a single location

In Figure 2 you can see that the logs are generated and saved in a single location in a database. The logs are generated using the Winston logging library and are in JSON format. The logs include

information about the API requests and responses, as well as any errors that may occur. They are also stored in a separate CSV file that is used for the LangChain framework to scan and analyze the logs. This might not be the most efficient, but for the purposes of this project where we are focused more on implementing LLM Log Analysis, this is sufficient.

Figure 3: Real-Time Logs in Terminal

In Figure 3 you can see the real-time logs that are generated in the terminal when the application is running. These logs provide information about the API requests and responses, as well as any errors that may occur. They are also used for debugging purposes during development.

The next steps will be to continue fine-tuning the integration of the LangChain framework and the Claude API to improve function, consistency and reliability. Once the application is able to consistently scan logs and identify anomalies, the next step will be to evaluate its performance using the evaluation metrics mentioned above.

6 Discussion and Challenges

7 Evaluation and Results

References

[1] Chris Egersdoerfer, Di Zhang, and Dong Dai. 2023. Early Exploration of Using ChatGPT for Log-based Anomaly Detection on Parallel File Systems Logs. In *Proceedings of the 32nd International Symposium on High-Performance Parallel and Distributed Computing* (Orlando, FL, USA) *(HPDC '23)*. Association for Computing Machinery, New York, NY, USA, 315–316. doi:10.1145/3588195.3595943

[2] Wei Guan, Jian Cao, Shiyou Qian, Jianqi Gao, and Chun Ouyang. 2025. LogLLM: Log-based Anomaly Detection Using Large Language Models. arXiv:2411.08561 [cs.SE] https://arxiv.org/abs/2411.08561

[3] Minghua He, Tong Jia, Chiming Duan, Huaqian Cai, Ying Li, and Gang Huang. 2024. LLMeLog: An Approach for Anomaly Detection based on LLM-enriched Log Events. In *2024 IEEE 35th International Symposium on Software Reliability Engineering (ISSRE)*. 132–143. doi:10.1109/ISSRE62328.2024.00023

[4] Jiaxing Qi, Shaohan Huang, Zhongzhi Luan, Shu Yang, Carol Fung, Hailong Yang, Depei Qian, Jing Shang, Zhiwen Xiao, and Zhihui Wu. 2023. LogGPT: Exploring ChatGPT for Log-Based Anomaly Detection. In *2023 IEEE International Conference on High Performance Computing & Communications, Data Science & Systems, Smart City & Dependability in Sensor, Cloud & Big Data Systems & Application (HPCC/DSS/SmartCity/DependSys)*. 273–280. doi:10.1109/HPCC-DSS-SmartCity-DependSys60770.2023.00045